

Optional Chaining (?.) - Simple Hinglish Summary

Source: <https://javascript.info/optional-chaining>

Main baat

Optional chaining (?.) nested object properties ko safely access karne ka tarika hai, chahe beech ki koi property exist na kare. (Ye recent addition hai, purane browsers me polyfill chahiye.)

1. "Non-existing property" ki problem

Maan lo (user) objects me users ki jaankari hai. Zyadatar users ka address (user.address) me hai aur street (user.address.street) me, lekin kuch ne address diya hi nahi.

Aise user ke liye (user.address.street) access karne par **error** aata hai:

```
javascript

let user = {}; // bina "address" property wala user

alert(user.address.street); // Error!
```

Ye expected hai: (user.address) (undefined) hai, aur (undefined) se (.street) lene par error aata hai. Lekin kai practical cases me hume **error ki jagah** (undefined) chahiye ("street nahi hai").

Ek aur example: web development me (document.querySelector('.elem')) element na milne par (null) deta hai:

```
javascript

// element na ho to null milta hai
let html = document.querySelector('.elem').innerHTML; // null ho to error
```

Kabhi kabhi element ka na hona normal hota hai, aur hume error nahi chahiye, bas (html = null).

Purane tarike (theek nahi)

(?) ternary se:

```
javascript
```

```
let user = {};  
  
alert(user.address ? user.address.street : undefined);
```

Kaam karta hai lekin `user.address` do baar likhna padta hai. `querySelector` ke case me to search do baar chalti hai, jo bura hai:

javascript

```
let html = document.querySelector('.elem') ? document.querySelector('.elem').in
```

Gehri nesting me aur bura ho jata hai:

javascript

```
alert(user.address ? user.address.street ? user.address.street.name : null : nu
```

Ye **bahut bura** aur samajhne me mushkil hai.

`&&` se (thoda behtar):

javascript

```
alert( user.address && user.address.street && user.address.street.name ); // un
```

Poore path ko AND karne se pakka hota hai ki har hissa exist karta hai. Lekin **property names abhi bhi duplicate** hote hain (`user.address` teen baar).

Isi liye **optional chaining** `?.` language me joda gaya, is problem ko hamesha ke liye khatam karne ke liye.

2. Optional chaining `?.`

`?.` tab evaluation rok deta hai jab uske pehle ki value `undefined` ya `null` ho, aur `undefined` return karta hai.

(Aage chhote me kehte hain ki koi cheez "exists" karti hai agar wo `null` aur `undefined` dono na ho.)

`value?.prop`:

- `value` exist karti hai to `value.prop` jaisa kaam karta hai
- Nahi to (`undefined/null` ho to) `undefined` return karta hai

javascript

```
let user = {}; // bina address

alert( user?.address?.street ); // undefined (error nahi)
```

Code chhota, saaf, aur koi duplication nahi.

`querySelector` ke saath:

javascript

```
let html = document.querySelector('.elem')?.innerHTML; // element na ho to unde
```

`user?.address` tab bhi kaam karta hai jab `user` object hi exist na kare:

javascript

```
let user = null;

alert( user?.address ); // undefined
alert( user?.address.street ); // undefined
```

Dhyan: `?.` sirf apne pehle wali value ko optional banata hai, uske aage ki nahi.

`user?.address.street.name` me `?.` sirf `user` ko `null/undefined` hone deta hai. Aage ki properties normal tarike se access hoti hain. Aur properties ko optional banana ho to unke `.` ko bhi `?.` se badalna padega.

`?.` ka zyada use mat karo

`?.` sirf wahan use karo jahan chiz ka na hona theek ho.

Jaise agar code ke logic ke hisaab se `user` hona hi chahiye lekin `address` optional hai, to likho `user.address?.street`, `user?.address?.street` nahi.

Tab agar `user` `undefined` ho gaya, to programming error dikhega aur hum use fix kar lenge. Warna `?.` ka zyada use **coding errors ko chupa deta hai**, aur debug karna mushkil ho jata hai.

`?.` se pehle wala variable declare hona chahiye

Agar `user` variable hi nahi hai, to `user?.anything` error deta hai:

javascript

```
// ReferenceError: user is not defined
user?.address;
```

Variable declare hona zaruri hai (`let/const/var user`) ya function parameter). Optional chaining sirf **declared variables** ke liye kaam karti hai.

3. Short-circuiting

`?.` left part exist na kare to evaluation **turant rok deta hai** ("short-circuit"). Isliye `?.` ke right side ke function calls ya operations **chalte hi nahi**.

```
javascript

let user = null;
let x = 0;

user?.sayHi(x++); // "user" nahi hai, isliye sayHi call aur x++ tak pahunchta h

alert(x); // 0, value nahi badhi
```

4. Dusre variants: `?.()`, `?.[]`

`?.` koi operator nahi, balki ek **special syntax construct** hai jo functions aur square brackets ke saath bhi kaam karta hai.

`?.()`: ek aisa function call karna jo exist na bhi kare.

```
javascript

let userAdmin = {
  admin() {
    alert("I am admin");
  }
};

let userGuest = {};

userAdmin.admin?.(); // I am admin

userGuest.admin?.(); // kuch nahi hota (method nahi hai)
```

Yahan dono lines me pehle dot se `admin` property li (`userAdmin.admin`), kyunki hum maante hain ki `user` object exist karta hai. Phir `?.()` left part check karta hai: `admin`

function ho to chal jata hai (`userAdmin`) ke liye), warna bina error ke rukk jata hai (`userGuest`) ke liye).

`?.[]`: jab dot ki jagah square brackets se property access karni ho:

```
javascript

let key = "firstName";

let user1 = {
  firstName: "John"
};

let user2 = null;

alert( user1?.[key] ); // John
alert( user2?.[key] ); // undefined
```

`delete` ke saath bhi:

```
javascript

delete user?.name; // user exist kare to user.name delete karo
```

Padhne aur delete karne ke liye, likhne ke liye nahi

`?.` assignment ke left side par kaam nahi karta:

```
javascript

let user = null;

user?.name = "John"; // Error, kaam nahi karta
// kyunki ye undefined = "John" ban jata hai
```

Summary

Optional chaining (`?.`) ke **teen forms** hain:

1. `obj?.prop` - `obj` exist kare to `obj.prop`, warna `undefined`
2. `obj?.[prop]` - `obj` exist kare to `obj[prop]`, warna `undefined`
3. `obj.method?.()` - `obj.method` exist kare to `obj.method()` call karta hai, warna `undefined`

`?.` left part ko `null/undefined` ke liye check karta hai, aur na ho to evaluation aage badhne deta hai. `?.` ki **chain** se nested properties safely access ho sakti hain.

Phir bhi `(?.)` **dhyan se** use karo, sirf wahan jahan code ke logic ke hisaab se left part ka na hona theek ho. Warna ye programming errors ko chupa dega.

Quick Summary

Syntax	Kya karta hai
<code>obj?.prop</code>	<code>obj</code> exist kare to <code>obj.prop</code> , warna <code>undefined</code>
<code>obj?.[key]</code>	Brackets ke saath same kaam
<code>obj.method?.()</code>	Method exist kare tabhi call karta hai
<code>delete obj?.prop</code>	Safe delete
<code>obj?.prop = 5</code>	Error (likhne me kaam nahi karta)

Yaad rakho:

- Sirf `(?.)` ke **pehle wali** value optional banti hai.
- Left part `null/undefined` ho to **short-circuit** ho jata hai (right side chalti hi nahi).
- Variable **declare** hona chahiye.
- **Zyada use mat karo**, warna asli bugs chhup jate hain.
- Aksar `(??)` ke saath jodkar default value dete hain: `user?.address?.street ?? "Unknown"`